

End-to-End Infrastructure (Python + Java) for a Licensed AI Data Marketplace

Upload → Quality Algorithms → Categorize/Store → Buy/Sell & Delivery

October 16, 2025

Abstract

This document specifies a pay-as-you-go, production-ready architecture using only **Python** and **Java**. It covers: a server-rendered Java website, secure direct-to-object-store uploads, Python-based quality & safety algorithms, classification/tagging, storage & indexing, and a compliant buying/selling flow with split payouts. It also defines security controls, NSFW/PII safeguards, observability, and minimal code skeletons in Python (FastAPI jobs) and Java (Spring Boot).

Contents

1	Constraints and Platform Choice	2
2	Components and Responsibilities	3
3	End-to-End Data Flow	3
4	Storage, Indexing, and Schema	4
4.1	Buckets and Object Layout (GCS)	4
4.2	Catalog (Postgres) — Core Tables	4
4.3	How Website Connects to Storage	5
5	Quality & Classification Pipeline (Python)	6
5.1	Key Packages (Python)	6
5.2	Two-Pass Algorithms (Summary)	6
5.3	Tagging & Classification	6
5.4	Python Job Skeleton (Evaluator)	6
6	Website: Java Spring Boot (Server-Side Rendered)	7
6.1	Why Spring MVC + Thymeleaf	7
6.2	Key Modules	7
6.3	Java Controller Sketches	7
7	Payments and Licenses	8
7.1	Stripe Connect (Java primary)	8
7.2	Licensing Artifacts	8
8	Safety (NSFW/PII) and Policy Gates	8
8.1	Automated Checks (Python)	8
8.2	Flow Control	8

9 Security, Identity, and Delivery	8
9.1 Identity and Access	8
9.2 Data Protection	8
9.3 Network and AppSec	8
10 Observability, Cost, and Ops	8
10.1 Observability	8
10.2 Cost Controls (Pay-as-you-go)	9
10.3 CI/CD and IaC	9
11 Sequence Summary (Textual)	9
12 Appendix A: Default Tooling	9
13 Appendix B: Example Policies (Illustrative)	10

1 Constraints and Platform Choice

Languages: Only **Python** and **Java** for backend, services, algorithms, and web server (server-side rendering).

Cloud: Google Cloud (GCP) pay-as-you-go:

- **Compute:** Cloud Run & Cloud Run Jobs (containerized Java/Spring Boot and Python/-FastAPI; autoscale to zero).
- **Storage:** Cloud Storage (GCS) buckets with versioning/lifecycle (raw, quarantine, curated, cards, samples).
- **Messaging:** Pub/Sub (event triggers).
- **DB:** Cloud SQL for Postgres (**pgvector** for embeddings).
- **Security:** Cloud KMS (CMEK), Secret Manager, Cloud Armor (WAF/rate limiting).
- **Delivery:** Cloud CDN in front of Cloud Run & signed URLs for GCS objects.

Regions: EU-first (e.g., **europe-west1**) for GDPR alignment.

2 Components and Responsibilities

Component	Language	Responsibility
Web App (Spring Boot MVC + Thymeleaf)	Java	Server-rendered website: auth, creator dashboard, catalog/search, dataset pages, checkout, download links
API Gateway (part of Web App)	Java	Issues signed-upload URLs, orchestrates orders/licensing, exposes REST endpoints for UI
Ingestor Job (Cloud Run Job)	Python	File-type/AV checks, checksums, EXIF/C2PA read, stage to quarantine/
Evaluator Job (Cloud Run Job, FastAPI worker)	Python	Quality/structure/info-theory/whiteness/NSFW/PII pipeline; writes Quality Card JSON; tags
Indexer Job	Python	Embeddings, LSH/perceptual dedupe signatures, writes Postgres + pgvector
Payments Service (Stripe Connect)	Java & Python (webhooks)	Checkout, split payouts, refunds; <i>primary integration in Java</i> ; Python consumer for async enrichment
Catalog DB (Cloud SQL: Postgres)	—	Datasets, versions, assets, tags, embeddings, orders, entitlements, audit
Object Store (GCS)	—	Buckets: raw/ , quarantine/ , curated/ , cards/ , samples/ ; signed URLs

3 End-to-End Data Flow

F1. Upload Init (Website → Signed URL)

- Authenticated creator opens “New Dataset” on the Java site.
- Java API creates a dataset/version row in Postgres, an *ingest manifest* (expected files,

sizes), and requests a **V4 signed URL** for each target object path in `raw/`.

- (c) Browser uploads directly to GCS using the signed URL(s) (no data passes through the web server).

F2. Object Finalize Event → Ingestor

- (a) GCS emits an `object.finalize` event → Pub/Sub topic with `dataset_id`, `version`, `paths`.
- (b) Python **Ingestor** Job (Cloud Run Job) is triggered; it validates MIME/size against the manifest, computes checksums, optional AV scan, extracts EXIF/C2PA, and moves files to `quarantine/`.

F3. Quality & Safety Evaluation (Two-Pass)

- (a) Python **Evaluator** consumes messages or is invoked by Ingestor on success.
- (b) *Pass 1 (fast)*: sample-based stats (missingness, LSH-dup), embedding-based coverage/RAES, leakage screens, info-theory (entropy, MI baseline, compressibility), quick NSFW/PII.
- (c) *Pass 2 (deep)*: full dedupe (ANN/graphs), shift tests (MMD/energy/C2ST), annotation metrics, modality probes (e.g., Δ PPL, WER, CLIP), whiteness tests on residuals.
- (d) Evaluator writes a JSON **Quality Card** to `cards/` and moves approved assets to `curated/`.

F4. Index & Categorize

- (a) Python **Indexer** computes embedding centroids, dedupe signatures; writes tags (`classification`, `text-pretraining`, `ASR`, etc.) and metrics into Postgres + `pgvector`.
- (b) Dataset version becomes `listed` if all policy gates pass; otherwise `needs_review`.

F5. Browse, Buy, Deliver

- (a) Buyers browse catalog (Java site queries Postgres; no direct object access). Samples are served via short-lived signed URLs to `samples/`.
- (b) Checkout via **Stripe** (Java): `PaymentIntent` → confirmation. Stripe webhook (Java or Python) finalizes order.
- (c) On success: create **entitlements** in Postgres; issue **short-lived signed URLs** for `curated/` assets; deliver license receipt + Quality Card.

4 Storage, Indexing, and Schema

4.1 Buckets and Object Layout (GCS)

- `raw/`: direct browser uploads (signed URLs).
- `quarantine/`: after Ingestor validation; not buyer-accessible.
- `curated/`: approved, sellable assets (per-version subpaths).
- `cards/`: JSON Quality Cards (immutable).
- `samples/`: small, redacted or watermarked previews.

Key format: `datasets/$dataset_id/$version/$partition/$sha256.ext`. Versioning enabled; lifecycle to Nearline/Archive for older versions.

4.2 Catalog (Postgres) — Core Tables

```
CREATE EXTENSION IF NOT EXISTS vector;  
  
CREATE TABLE datasets (
```

```

    id UUID PRIMARY KEY,
    owner_id UUID NOT NULL,
    name TEXT NOT NULL,
    modality TEXT CHECK (modality IN ('text','image','audio','tabular')),
    visibility TEXT CHECK (visibility IN ('private','listed','delisted'))
        DEFAULT 'private',
    license_preset TEXT, territory TEXT[],
    created_at TIMESTAMPTZ DEFAULT now(), updated_at TIMESTAMPTZ DEFAULT
        now()
);

CREATE TABLE dataset_versions (
    id UUID PRIMARY KEY,
    dataset_id UUID REFERENCES datasets(id),
    v INT NOT NULL, status TEXT NOT NULL, -- 'listed', 'needs_review', '
        blocked'
    quality_overall NUMERIC, grade TEXT, card_uri TEXT,
    created_at TIMESTAMPTZ DEFAULT now(), UNIQUE(dataset_id, v)
);

CREATE TABLE assets (
    id UUID PRIMARY KEY,
    dataset_version_id UUID REFERENCES dataset_versions(id),
    path TEXT NOT NULL, bytes BIGINT, sha256 TEXT, mime TEXT, partition
        TEXT
);

CREATE TABLE tags (
    dataset_version_id UUID REFERENCES dataset_versions(id),
    tag TEXT, score NUMERIC, PRIMARY KEY (dataset_version_id, tag)
);

CREATE TABLE embeddings ( -- e.g., 384-dim CLIP/text embeddings
    dataset_version_id UUID REFERENCES dataset_versions(id),
    name TEXT, vec vector(384),
    PRIMARY KEY (dataset_version_id, name)
);

CREATE TABLE orders (
    id UUID PRIMARY KEY, buyer_id UUID, total_cents BIGINT, currency TEXT
        ,
    stripe_payment_intent TEXT, status TEXT, created_at TIMESTAMPTZ
        DEFAULT now()
);

CREATE TABLE entitlements (
    order_id UUID REFERENCES orders(id),
    dataset_version_id UUID REFERENCES dataset_versions(id),
    rights TEXT, expires_at TIMESTAMPTZ, PRIMARY KEY (order_id,
        dataset_version_id)
);

```

4.3 How Website Connects to Storage

The Java site **never** reads GCS directly. It:

- (a) Queries Postgres for metadata, tags, scores, card URIs.
- (b) Requests *short-lived signed URLs* from a backend endpoint for previews/downloads.

(c) Renders links that expire (e.g., 10–15 minutes), optionally via Cloud CDN.

5 Quality & Classification Pipeline (Python)

5.1 Key Packages (Python)

Core: numpy, scipy, pandas, scikit-learn, statsmodels.

Expectations & drift: great_expectations, evidently.

Similarity/dedup: datasketch (MinHash), faiss or hnswlib, imagehash, opencv-python.

Text: transformers (small models), perplexity probes; presidio (PII); cleanlab (label noise).

Audio: librosa, jiwer (WER). **Images:** CLIP via transformers.

Info-theory: KSG MI (custom or pyitlib), compression signals.

Safety: open-source NSFW models (LAION/NudeNet), profanity lists; thresholded policies.

5.2 Two-Pass Algorithms (Summary)

Pass 1: schema & checksums → sample embeddings → missingness/dup/coverage/RAES → leakage screens → entropy/entropy-rate/MI baselines/compressibility → quick NSFW/PII.

Pass 2: full dedupe (ANN/graphs) → shift tests (MMD/energy/C2ST) → annotation κ/α → modality probes (Δ PPL, WER, CLIP diversity, sharpness) → whiteness tests (Ljung–Box, BDS, SFM).

5.3 Tagging & Classification

Rules + meta-model convert metrics to tags:

- text-pretraining, RAG, classification, ASR, vision-pretraining, etc.
- Scores (0–1) stored in tags table; thresholds drive listing eligibility and search ranking.

5.4 Python Job Skeleton (Evaluator)

```
def evaluate_and_index(dataset_id: str, version: int, gcs_paths: list[
    str]):
    # 1) Load sample for Pass 1
    files = sample_paths(gcs_paths, n= min(10000, len(gcs_paths)))
    meta = infer_schema_and_checksums(files)
    z = embed(files)                                # modality-aware small
                                                    model
    stats = compute_stats(files, z)                  # missing, dup(LSH),
                                                    RAES, coverage, etc.
    info = info_signals(files, z)                    # entropy, MI,
                                                    compressibility, NCD
    risk = quick_safety(files)                        # PII/NSFW/toxicity,
                                                    prevalence-calibrated

    deep = {}
    if should_run_deep(stats, risk, info, total=len(gcs_paths)):
        deep["dedupe"] = full_dedupe(gcs_paths)
        deep["shift"] = shift_tests(gcs_paths, z)
        deep["annq"] = annotation_quality(files)
        deep["probes"] = modality_probes(files)
        deep["white"] = whiten_residuals(files)

    scores, tags = aggregate_scores_and_tags(meta, stats, info, risk,
        deep)
```

```

card_uri = write_quality_card_to_gcs(dataset_id, version, scores,
    tags, diagnostics=merge(...))
move_assets_to_curated_if_pass(dataset_id, version, policy=scores)
upsert_postgres(dataset_id, version, tags, card_uri, embeddings=
    centroid(z))

```

6 Website: Java Spring Boot (Server-Side Rendered)

6.1 Why Spring MVC + Thymeleaf

- Satisfies “Python + Java only” constraint; no Node/TS build chain.
- SEO-friendly SSR, minimal client JS (only for Stripe Checkout).

6.2 Key Modules

- **Auth:** Spring Security (OAuth2 login), session management.
- **Creator:** create dataset, request signed upload URLs, view pipeline status.
- **Catalog:** search/filter (tags, modality, quality), dataset pages with Quality Card.
- **Checkout:** Stripe Checkout (stripe-java); orders & entitlements.
- **Downloads:** endpoint issues short-lived signed URLs after entitlement check.

6.3 Java Controller Sketches

Signed Upload URL

```

@PostMapping("/api/upload/signed-url")
public SignedUrlResponse getSignedUrl(@RequestBody UploadInit req,
    Principal user) {
    // 1) authorize & create dataset/version rows in Postgres
    DatasetVersion v = catalogService.initVersion(req, user);
    // 2) compute GCS object path(s)
    String objectPath = String.format("datasets/%s/%d/%s", v.datasetId(),
        v.v(), req.filename());
    // 3) use GCS V4 signer (service account) to generate a short-lived
        signed URL
    URL signedUrl = gcsSigner.signV4(objectPath, req.contentType(),
        Duration.ofMinutes(15));
    return new SignedUrlResponse(signedUrl.toString(), objectPath);
}

```

Issue Short-Lived Download Link

```

@PostMapping("/api/downloads")
public DownloadLink issueDownload(@RequestBody DownloadReq req,
    Principal user) {
    // 1) check entitlements in Postgres for (user, datasetVersion)
    if (!entitlementsService.allowed(user, req.datasetVersionId())) {
        throw new ResponseStatusException(HttpStatus.FORBIDDEN);
    }
    // 2) sign URL for curated object (TTL 10-15 min)
    URL url = gcsSigner.signV4(req.objectPath(), "application/octet-
        stream",
        Duration.ofMinutes(15));
    return new DownloadLink(url.toString(), Instant.now().plusSeconds
        (900));
}

```

7 Payments and Licenses

7.1 Stripe Connect (Java primary)

- Onboarding: Connect accounts for creators (KYB/KYC).
- Checkout: `PaymentIntent` + Checkout Session; upon success, create `orders` + `entitlements`.
- Payouts: platform fee retained, creator share scheduled (e.g., T+7).
- Webhooks: Java (primary) handles `payment_intent.succeeded/checkout.session.completed`; Python consumer can enrich logs.

7.2 Licensing Artifacts

- License presets (train-only, eval-only, redistribution/no-redistribution, territory).
- Each order produces a **license receipt** (JSON/PDF) with dataset version, hashes, timestamps, and license terms.

8 Safety (NSFW/PII) and Policy Gates

8.1 Automated Checks (Python)

- **Text:** `presidio` NER + pattern rules for PII; toxicity/profanity classifiers.
- **Images:** NSFW classifier; face detection; EXIF/C2PA provenance.
- **Audio:** VAD segmentation; profanity; SNR thresholds; diarization hints.

Medical/Research Exceptions: gated category + stricter license; higher reviewer approval.

8.2 Flow Control

- High-severity flags $\Rightarrow$ `needs_review` (manual queue).
- Blocklisting for repeat infringers; tamper-evident evidence (hashes, sample fingerprints).

9 Security, Identity, and Delivery

9.1 Identity and Access

- Spring Security OAuth2 (users); IAM service accounts (jobs).
- Role-based ACLs: owner, collaborator, reviewer, buyer; entitlement checks per request.

9.2 Data Protection

- CMEK via Cloud KMS for GCS & Cloud SQL; TLS 1.2+; HSTS.
- Signed URLs (short TTL); per-download logging; optional download caps by buyer.

9.3 Network and AppSec

- Cloud Armor WAF + rate limits; Private Serverless VPC; Cloud SQL private IP.
- Secret Manager; no secrets in images; rotation policies.

10 Observability, Cost, and Ops

10.1 Observability

- Cloud Logging/Monitoring + OpenTelemetry traces; Sentry for Java/Python.
- SLOs: API p95 < 300 ms; ingest start < 30 s; quality TTR < 30 min for 10 GB.

10.2 Cost Controls (Pay-as-you-go)

- Cloud Run autoscale to zero; per-second billing.
- Lifecycle: older versions to Nearline/Archive; deep-pass sampling caps.
- Quotas per tenant; batch heavy jobs off-peak.

10.3 CI/CD and IaC

- Build with **Maven/Gradle** (Java) and **Poetry/pip** (Python); Docker images to Artifact Registry.
- Terraform modules for IAM, Cloud Run, Pub/Sub, GCS, Cloud SQL, KMS.
- Blue/green deploys for Web App; Job image pinning with digests in Quality Card.

11 Sequence Summary (Textual)

- S1: Create listing (Java):** create dataset/version, get signed upload URL(s).
S2: Upload (Browser→GCS): direct PUT to `raw/` with V4 signed URL.
S3: Finalize (Event): GCS → Pub/Sub → Python Ingestor.
S4: Ingestor (Python): MIME/checksums/EXIF/C2PA; move to `quarantine/`.
S5: Evaluator (Python): two-pass quality/safety; write Quality Card; move to `curated/`.
S6: Indexer (Python): embeddings/tags to Postgres + `pgvector`; version → `listed`.
S7: Browse/Buy (Java): render catalog; Stripe Checkout; on success create entitlements.
S8: Deliver (Java): issue short-lived signed URLs for `curated/` assets; log download.

12 Appendix A: Default Tooling

Area	Lang	Tools/Packages
Web server	Java	Spring Boot MVC, Thymeleaf, Spring Security (OAuth2)
Payments	Java	stripe-java SDK, webhook controller
API client	Java	JDBC + jOOQ (Postgres), pgjdbc-ng
Jobs	Python	FastAPI worker (for control), Cloud Run Jobs endpoints
Quality	Python	numpy, pandas, scikit-learn, statsmodels, great_expectations
Similarity	Python	datasketch, faiss/hnswlib, imagehash, opencv-python
Text	Python	transformers (small), presidio, cleanlab
Audio	Python	librosa, jiwer
Images	Python	CLIP (transformers)
Info-theory	Python	pyitlib (or custom KSG), gzip/brotli for MDL proxies
Observability	Java/Py	OpenTelemetry, Sentry
Infra	—	Terraform; Cloud KMS, Secret Manager, Cloud Armor

13 Appendix B: Example Policies (Illustrative)

Signal	Threshold/Rule	Action
Text PII score	$> T_{\text{PII}}$ on samples	Quarantine + de-ID required; manual review
Image NSFW prob	> 0.8 (no medical flag)	Block listing; allow appeal with evidence
Audio profanity	ratio $> r$	Gated tag + license restriction
Child safety	any positive	Immediate block + escalation
Duplication rate	$> 10\%$	Reduce or relabel; quality downgrade
Leakage	$\text{MI}(\text{split}; \text{features}) > 2\sigma$ baseline	Resplit or reject; flag in Card

Summary. This architecture keeps the stack to **Python + Java** while delivering a secure, pay-as-you-go marketplace. Data flows directly to object storage with signed URLs; Python jobs compute quality & tags; Java serves the website, search, checkout, and short-lived download links; storage is versioned and policy-gated; and all access is auditable and least-privileged.